

GoldenHour

One tap. Two lifelines. Inside the hour that decides.

MVP | Technical Documentation

Emergency Hospital Routing & Replacement-Blood Alerting

A GPS-triggered emergency response system that finds the right hospital *and* arranges compatible blood — two parallel actions from a single tap, working in any Indian city.

Event	Bharat Academix CodeQuest 2026 — Prototype / MVP Round
Team	Team GoldenHour
Category	Healthcare Technology ▪ Social Impact ▪ Emergency Response
Repository	https://github.com/SarmaHighOnCode/GoldenHour
Stack	React 19 PWA ▪ FastAPI ▪ Supabase / PostGIS ▪ Google Maps

Contents

1	Executive Summary	1
2	Problem Statement & Motivation	2
2.1	The golden hour, and where it is lost	2
2.2	The user we build for	2
2.3	Why existing solutions fall short	2
3	Solution Overview	3
3.1	Key differentiators	3
4	System Architecture	3
4.1	Design rules that keep it clean	3
4.2	Two interchangeable stores	4
5	Core Algorithms	4
5.1	Hospital ranking	4
5.2	Blood matching	4
5.3	Confirmation race resolution	5
6	API Design	5
6.1	Representative request / response	5
7	Frontend & User Experience	6
7.1	Live updates	6
7.2	Accessibility & resilience	6
8	Technology Stack	6
9	Reliability, Security & Performance	6
10	Scalability	7
11	Testing & Continuous Integration	7
12	Deployment	7
13	Scope & Future Work	8
14	Conclusion	8
A	Appendix — Configuration Reference	8

1 Executive Summary

GoldenHour is a Progressive Web App with a feature-phone SMS fallback that converts a single GPS-triggered emergency into *two simultaneous actions*: it locates and confirms a hospital with the correct department and a free bed, *and* alerts nearby compatible replacement-blood donors — both inside the “golden hour” after a trauma, when minutes decide outcomes.

The system is built for the reality of Indian emergency care: the majority of patients reach hospital not by ambulance but by private car or auto, entirely outside any dispatch system. Today

that family loses critical minutes driving to a hospital that turns out to have no bed or no matching department, and arranges blood through frantic phone calls. GoldenHour spends those same minutes finding a hospital that can *actually take the patient* and broadcasting a structured blood request — automatically, from one screen.

What is delivered in this MVP:

- A working full-stack application — React 19 PWA + FastAPI service — deployed live.
- Three production algorithms: hospital ranking, ABO/Rh blood matching, and first-to-accept confirmation.
- Real, live hospital data fetched per-location (OpenStreetMap + Google Places) — not seeded mock data.
- A clean two-store data layer that runs the demo with *zero external services* and swaps to Supabase + PostGIS for production with no code changes.
- A test suite (31 passing) covering algorithms, the API contract, the live flow, and the production database path; CI on Python 3.11 and 3.12.

2 Problem Statement & Motivation

2.1 The golden hour, and where it is lost

In trauma medicine, the first hour after injury — the *golden hour* — is when timely care most strongly determines survival. In India, that hour is routinely lost not because care does not exist, but because:

1. **The right hospital is not found fast enough.** A family drives to the nearest or best-known hospital, which may lack the needed department (trauma, cardiac, obstetric) or have no free bed — then drives again.
2. **Replacement blood is arranged too late.** Surgery draws down the blood bank's licensed supply; restocking is organised through ad-hoc phone calls, often after the patient is already on the table.

2.2 The user we build for

Most existing tools assume an ambulance and a dispatcher. GoldenHour targets the **self-transporting family** — the bystander or relative holding a phone at the roadside. They need one action, not four apps.

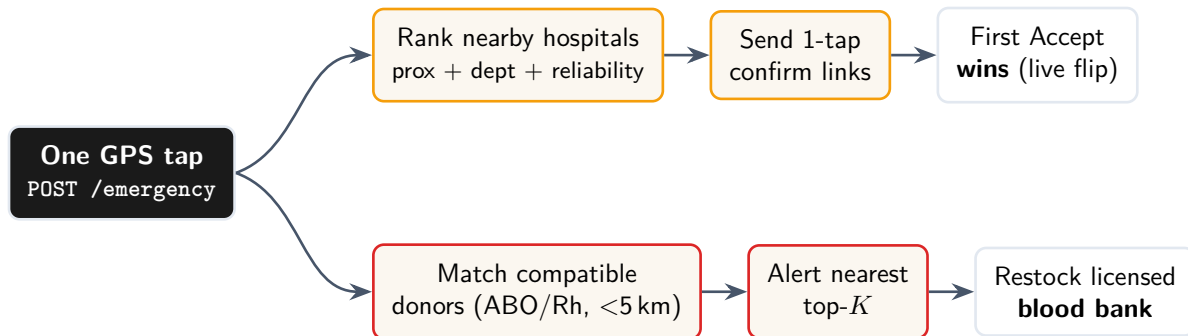
2.3 Why existing solutions fall short

What exists	The gap GoldenHour closes
Blood-donor apps (e-RaktKosh, Friends2Support, Blood-Connect)	Solve <i>only</i> blood, never routing, and often direct donors to the patient — medically wrong.
Hospital / ambulance finders (108, Practo)	Show a <i>list</i> ; no knowledge of whether a bed is actually free, no blood path.
General maps apps	Distance only — no department match, no confirmation, no concurrency.

No widely available tool runs hospital routing *and* blood matching *concurrently* from a single trigger, with a confirmation loop that never shows a bed that is not real. That concurrency, plus the correct *replacement-donation* model, is the core contribution.

3 Solution Overview

A single `POST /emergency` call fans out into two parallel branches:

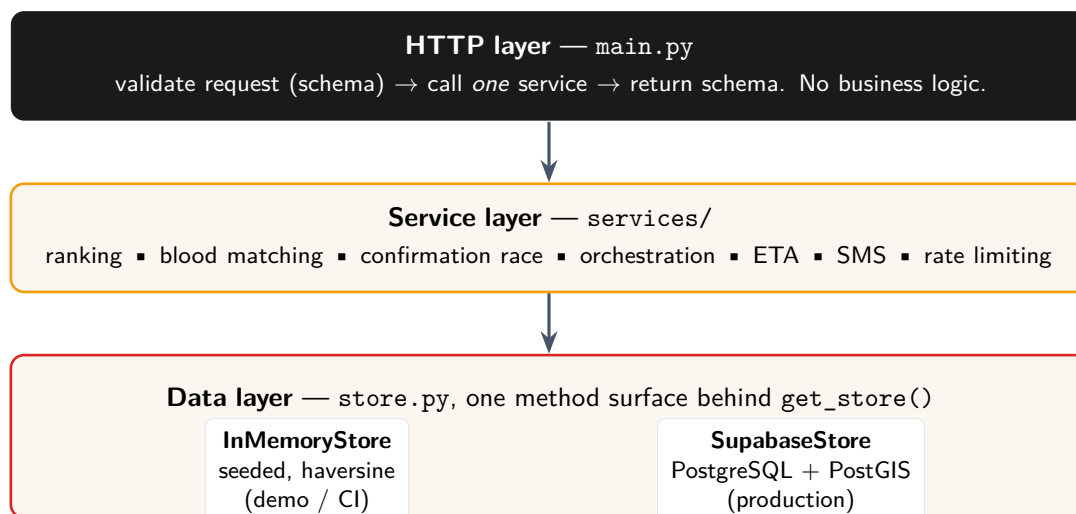


3.1 Key differentiators

- **Two lifelines, one tap, in parallel.** Routing and blood run concurrently inside one request — not two separate apps.
- **Never a false bed.** The first hospital to *accept* takes the patient; later acceptances are told it is already routed. The UI never shows a “bed available” that is not real.
- **Medically correct blood model.** Donors are sent to the hospital’s *licensed blood bank* for *replacement* donation that restocks supply — not to the patient, which is medically useless.
- **Works in any Indian city.** Hospitals are fetched live from OpenStreetMap (Overpass) and Google Places around the patient’s actual coordinates — the data is not seeded.
- **Graceful degradation.** Feature-phone SMS path, an unconfirmed-response fallback to 1-tap calling, and a demo mode that needs no external services at all.

4 System Architecture

GoldenHour is a two-part system split by ownership and joined by one contract (`API_CONTRACT.md`). The frontend never touches the database; it only calls the API. The backend is a strict three-layer design with two interchangeable data stores behind a single factory.



4.1 Design rules that keep it clean

- **Routes are thin.** Each handler validates input with a Pydantic schema, calls exactly one service function, and returns a schema.

- **Services never touch HTTP.** They take a `store` plus plain values and return plain dicts — trivially unit-testable without a client.
- **All data access goes through `store.py`.** Its method surface (`compatible_donors_nearby` , `create_emergency` , ...) is identical for both stores.
- **Schemas are the contract.** `schemas.py` mirrors `API_CONTRACT.md` field-for-field; the frontend builds against the same shapes.

4.2 Two interchangeable stores

The radius and scoring mathematics are identical whether they run in Python (haversine) or in PostGIS (`ST_DWithin` / `ST_Distance`). `get_store()` selects Supabase when `SUPABASE_URL` and a key are present and reachable, and falls back to the in-memory store otherwise — with automatic fallback if the client cannot be created, so boot never hard-fails. Consequences:

- The demo runs end-to-end with no external services.
- CI needs no database.
- The production path (`sql/schema.sql` + `seed_supabase.py`) is a true drop-in — no service-code changes.

5 Core Algorithms

5.1 Hospital ranking

For the department the emergency needs, every nearby hospital is scored and the top N (default 5) are returned, best-first:

$$P(h) = 0.5 \cdot \text{prox}(h) + 0.3 \cdot \text{dept}(h) + 0.2 \cdot \text{rel}(h)$$

$$\text{prox}(h) = \text{clamp}\left(1 - \frac{\text{eta}}{\text{eta}_{\max}}, 0, 1\right) \quad \text{closer is better}$$

$$\text{dept}(h) = \begin{cases} 1 & \text{needed department present} \\ 0 & \text{otherwise} \end{cases}$$

$$\text{rel}(h) = \text{live acceptance rate} \quad \text{historically reliable}$$

Reliability cold start. $\text{rel}(h)$ enters the score only once a hospital has logged at least `REL_ACTIVATION_THRESHOLD` (default 20) confirmations — before that there is no honest reliability signal. Until then the score degrades to proximity + department, renormalised by 0.8 so it still spans $[0, 1]$:

$$P(h) = \frac{0.5 \cdot \text{prox}(h) + 0.3 \cdot \text{dept}(h)}{0.8}.$$

At launch (no confirmations yet) ranking is therefore pure proximity + department. The ETA for every candidate is fetched *concurrently* (see §9), so latency stays flat as the candidate pool grows.

5.2 Blood matching

Given the requested group, the matcher selects donors who are (a) of a *compatible* group per a hardcoded ABO/Rh table, (b) available, (c) within the search radius (default 5 km), and (d) off their post-donation cooldown — *sex-aware*: 90 days for men, 120 for women, per national guidelines (`last_donated IS NULL OR last_donated < today - cooldown`). The nearest top- K are alerted with directions to the hospital's **licensed blood bank**. Rh-negative requests are flagged `rare_group` so the UI can warn that supply may be scarce.

5.3 Confirmation race resolution

Each ranked hospital receives a single-use one-tap link. The **first hospital to accept** takes the patient; any later acceptance receives `already_confirmed: true` and does not override the winner. The accepted hospital flips `pending` → `confirmed`, which the patient's screen sees via Supabase Realtime (or polling in demo mode). If nothing confirms within `UNCONFIRMED_FALLBACK_SECONDS` (default 180), the status response flags `unconfirmed_fallback` so the UI surfaces the nearest hospitals with a 1-tap call — *never* a false “bed available”.

6 API Design

The API is the single source of truth both sides build against; interactive docs are auto-generated at `/docs` (Swagger) and `/redoc`.

Method & path	Purpose
POST <code>/emergency</code>	Trigger — rank hospitals, alert donors, send confirmation links.
GET <code>/emergency/{id}/status</code>	Live status — hospital replies + donor responses (poll or Realtime).
GET <code>/confirm/{token}</code>	Emergency details shown on the hospital's confirmation page.
POST <code>/confirm/{token}</code>	Hospital taps Accept (<code>true</code>) / Not Available (<code>false</code>).
GET <code>/donor/respond/{token}</code>	Donor one-tap response (records intent, returns an HTML thank-you).
POST <code>/donor/register</code>	Register a replacement-blood donor (optional <code>sex</code> sets cooldown).
POST <code>/sms/inbound</code>	Feature-phone path — parse an SMS, reply with nearest hospitals.
GET <code>/health</code> • GET <code>/ready</code>	Liveness (mode) • readiness (pings the data layer).
GET <code>/dev/links</code> <code>/dev/alerts</code>	• Demo helpers — links sent to hospitals • alerts sent to donors.

6.1 Representative request / response

Listing 1: POST `/emergency` — request and response

```

1 // Request
2 { "lat": 26.9124, "lng": 75.7873, "emergency_type": "trauma", "blood_group": "0+" }
3
4 // Response (hospitals already sorted best-first by the backend)
5 {
6   "request_id": "abc123",
7   "hospitals": [
8     { "hospital_id": "h1", "name": "SMS Hospital", "lat": 26.9036, "lng": 75.8147,
9       "eta_minutes": 6, "department_match": true, "distance_km": 2.3,
10      "status": "pending", "phone": "+91xxxxxxxxxx" }
11   ],
12   "donors_alerted": 5,
13   "rare_group": false
14 }
```

The write endpoints are rate-limited per client (fixed-window); coordinates outside valid ranges are rejected with `422`; unknown tokens return `404`.

7 Frontend & User Experience

The PWA is a mobile-first React 19 application (Vite + Tailwind) with four screens:

- **Patient Intake** — one-tap GPS capture, emergency type and blood group.
- **Live Results** — ranked hospital cards (ETA, department match, status), an interactive Leaflet map with the patient pin, hospital pins and route lines, a Golden-Hour countdown ring, and a donor-response panel.
- **Hospital Confirm** — the one-tap Accept / Not Available page a hospital opens.
- **Donor Registration** — name, phone, group, location, last-donated date.

7.1 Live updates

The results screen subscribes to Supabase Realtime for instant `pending` → `confirmed` flips, with a 3-second polling loop as a fallback if the socket drops. On confirmation the map auto-frames the route to the winning hospital and a success banner slides in.

7.2 Accessibility & resilience

All animation respects `prefers-reduced-motion` (static fallbacks for the map ping, route lines and card transitions). The UI degrades cleanly to call links when no hospital confirms, and the data layer's normalisation guards tolerate ETA values arriving as either minutes or seconds.

8 Technology Stack

Layer	Technology
Frontend	React 19, Vite, Tailwind CSS, React Router, Framer Motion, Leaflet, Supabase JS client
Backend	Python 3.12, FastAPI, Uvicorn, Pydantic v2
Database	PostgreSQL + PostGIS (via Supabase), Supabase Realtime
Maps / ETA	Google Maps Distance Matrix; Google Places + OpenStreetMap (Overpass) for hospital discovery
Messaging	Console / Telegram (demo), MSG91 / Gupshup + DLT (production)
Infra	Docker, Render (API), Vercel (PWA), GitHub Actions CI

9 Reliability, Security & Performance

- **Concurrent external calls.** Per-candidate ETA lookups run under `asyncio.gather`, so a busier city with more candidates does not increase response latency linearly — it stays bounded by the slowest single call.
- **Rate limiting.** Fixed-window per-client limits guard every public write endpoint (`/emergency`, `/confirm`, `/donor/register`, `/donor/respond`); excess returns `429`. The limiter honours `X-Forwarded-For` behind the platform proxy.
- **Single-use tokens.** Confirmation and donor-response links are URL-safe tokens, single-use; no PII beyond a phone number; secrets never reach the client (logging suppresses URLs that contain keys).
- **Health & readiness.** `/health` is a cheap liveness probe that also accepts `HEAD` (for uptime monitors); `/ready` pings the data layer and returns `503` if the database is unreachable — so a

deploy can verify it is truly connected to Supabase and not silently in demo mode.

- **Graceful degradation.** Overpass mirrors are raced with timeouts; the unconfirmed-response fallback surfaces call links; the SMS path serves feature phones; and demo mode keeps the product fully functional with no dependencies.

10 Scalability

- **Stateless API.** The service holds no per-request session state, so it scales horizontally behind a load balancer; rate-limit state is the only in-process concern and is designed to back onto Redis for a multi-node deploy.
- **Geo at the database.** In production, donor-radius queries become real PostGIS `ST_DWithin` lookups with spatial indexes — the same maths as the demo's haversine, but executed where the data lives.
- **Realtime instead of polling.** Confirmations stream to the patient over Supabase Realtime, removing poll load at scale.
- **Concurrency by default.** Hospital discovery and ETA computation are already concurrent, so latency is governed by the slowest single upstream, not the candidate count.
- **On-demand, cached discovery.** Hospitals for a new region are fetched once from OSM/Places and cached per region, so the system works in *any* city without pre-seeding all of India.

11 Testing & Continuous Integration

The suite (31 tests) runs fully offline — no Supabase, no network — and verifies:

- the exact `API_CONTRACT.md` response shapes;
- the ranking, blood-matching and confirmation algorithms;
- the end-to-end emergency → confirm → status-flip flow;
- the **production SupabaseStore path** against a fake Supabase client (`tests/fake_supabase.py`), so column names, query chains and key mapping are validated without a live database.

CI (GitHub Actions) runs the same suite on Python 3.11 and 3.12, with lint (`ruff`) and format gates enforced on every push.

12 Deployment

Component	Platform	Configuration
Backend API	Render (or Fly.io)	<code>render.yaml</code> — Python runtime, <code>rootDir: backend</code> , <code>uvicorn main:app</code> .
Frontend PWA	Vercel	<code>vercel.json</code> — builds <code>frontend/</code> , SPA rewrites.
Database	Supabase	Run <code>sql/schema.sql</code> , enable Realtime on <code>confirmation_requests</code> .

Production secrets (`SUPABASE_*`, `GOOGLE_MAPS_API_KEY`, `MSG91_AUTH_KEY`, `VITE_API_BASE_URL`) live in the respective dashboards and are never committed. The same Docker image runs locally, in CI and in production.

13 Scope & Future Work

Deliberately out of scope for the MVP (integration work, not the hard problem): real SMS gateway billing / DLT registration, hospital-staff authentication, and ambulance dispatch integration. The hard problem — concurrent ranking + blood matching that works in any city, with a confirmation loop that never lies about a bed — is what is built and verified here.

Roadmap:

- Live blood-bank inventory integration (replace synthetic availability with real stock).
- Hospital-side dashboard and authenticated accept/decline.
- Production SMS with DLT-registered templates for true feature-phone reach.
- Redis-backed rate limiting and multi-region deployment.

14 Conclusion

GoldenHour turns the single most time-critical decision in an emergency — where to go, and how to get blood ready — into one tap that does both, in parallel, anywhere in India. The engineering is honest: real data, a contract-driven architecture, a demo that needs nothing, and a production path that is a configuration switch rather than a rewrite. It is not a directory and not a donor list; it is a dispatch system for the hour that decides.

A Appendix — Configuration Reference

Variable	Default	Effect
<code>SUPABASE_URL</code> + key	—	Set → use the real database (else demo).
<code>GOOGLE_MAPS_API_KEY</code>	—	Set → real drive-time ETAs and Places discovery.
<code>FRONTEND_URL</code>	localhost:5173	Base for <code>/confirm/<token></code> links.
<code>DONOR_RADIUS_METERS</code>	5000	Donor search radius.
<code>DONOR_COOLDOWN_DAYS</code>	90	Post-donation cooldown (men / default).
<code>DONOR_COOLDOWN_DAYS_FEMALE</code>	120	Post-donation cooldown (women).
<code>DONOR_ALERT_K</code>	10	Nearest matched donors actually alerted.
<code>REL_ACTIVATION_THRESHOLD</code>	20	Confirmations before <code>rel(h)</code> counts.
<code>UNCONFIRMED_FALLBACK_SECONDS</code>	180	No-confirmation window before the 1-tap-call fallback.
<code>RANKING_TOP_N</code>	5	Hospitals returned per emergency.